VAUTRIN OS

Software Design Documentation Pack — v0.1

SDDs • ADRs • AGENTS.md • Technical Backlog

Initial vertical: Remodeling & Home Services — United States

Architecture baseline: Multi-tenant modular monolith, monorepo, Nuxt 4, NestJS, PostgreSQL + pgvector, Redis/BullMQ.

Document Index

Section	Document
README	README.md
AGENTS.md	AGENTS.md
SDD — 00-overview.md	docs/sdd/00-overview.md
SDD — 01-requirements.md	docs/sdd/01-requirements.md
SDD — 02-architecture.md	docs/sdd/02-architecture.md
SDD — 03-domain-model.md	docs/sdd/03-domain-model.md
SDD — 04-database.md	docs/sdd/04-database.md
SDD — 05-multitenancy.md	docs/sdd/05-multitenancy.md
SDD — 06-ai-system.md	docs/sdd/06-ai-system.md
SDD — 07-workflow-engine.md	docs/sdd/07-workflow-engine.md
SDD — 08-integrations.md	docs/sdd/08-integrations.md
SDD — 09-security.md	docs/sdd/09-security.md
SDD — 10-observability.md	docs/sdd/10-observability.md
SDD — 11-testing.md	docs/sdd/11-testing.md
SDD — 12-deployment.md	docs/sdd/12-deployment.md
ADR — 001-monorepo-strategy.md	docs/adr/001-monorepo-strategy.md
ADR — 002-modular-monolith.md	docs/adr/002-modular-monolith.md
ADR — 003-multi-tenancy.md	docs/adr/003-multi-tenancy.md
ADR — 004-postgresql-pgvector.md	docs/adr/004-postgresql-pgvector.md
ADR — 005-tenant-isolation-rls.md	docs/adr/005-tenant-isolation-rls.md
ADR — 006-ai-provider-abstraction.md	docs/adr/006-ai-provider-abstraction.md
ADR — 007-workflow-engine.md	docs/adr/007-workflow-engine.md
ADR — 008-event-driven-processing.md	docs/adr/008-event-driven-processing.md
ADR — 009-queue-strategy.md	docs/adr/009-queue-strategy.md
ADR — 010-integration-abstraction.md	docs/adr/010-integration-abstraction.md
ADR — 011-authentication-rbac.md	docs/adr/011-authentication-rbac.md
ADR — 012-observability.md	docs/adr/012-observability.md
Backlog — Technical Backlog	backlog/technical-backlog.md

README.md

Vautrin OS Documentation — v0.1

Vautrin OS is a multi-tenant SaaS platform for local businesses, initially verticalized for remodeling and home-service businesses in the United States.

This documentation package is the architectural baseline for MVP development. It is intentionally written in English so it can be consumed directly by human developers and AI coding agents.

Product principle

Sell operational outcomes, not AI technology.

The MVP focuses on:

> Lead capture → AI reception → qualification → follow-up → appointment → estimate → estimate follow-up.

The architecture is vertical-agnostic even though the first production template is `remodeling`.

Architecture baseline

- Monorepo: pnpm workspaces + Turborepo
- Frontend: Nuxt 4 + Vue 3 + TypeScript
- Backend: NestJS + TypeScript
- Architecture: Modular monolith
- Database: PostgreSQL
- Vector search: pgvector
- Async processing: Redis + BullMQ
- AI: provider-agnostic LLM abstraction
- CI/CD: GitHub Actions
- Deployment: containerized
- Multi-tenancy: pooled model with application isolation + PostgreSQL RLS
- Observability: structured logs + tracing/metrics-ready interfaces
- Testing: unit + integration + E2E + tenant-isolation + AI evaluation

Documentation map

SDD

- `docs/sdd/00-overview.md`
- `docs/sdd/01-requirements.md`
- `docs/sdd/02-architecture.md`
- `docs/sdd/03-domain-model.md`
- `docs/sdd/04-database.md`

- `docs/sdd/05-multitenancy.md`
- `docs/sdd/06-ai-system.md`
- `docs/sdd/07-workflow-engine.md`
- `docs/sdd/08-integrations.md`
- `docs/sdd/09-security.md`
- `docs/sdd/10-observability.md`
- `docs/sdd/11-testing.md`
- `docs/sdd/12-deployment.md`

ADR

- `docs/adr/001-monorepo-strategy.md`
- `docs/adr/002-modular-monolith.md`
- `docs/adr/003-multi-tenancy.md`
- `docs/adr/004-postgresql-pgvector.md`
- `docs/adr/005-tenant-isolation-rls.md`
- `docs/adr/006-ai-provider-abstraction.md`
- `docs/adr/007-workflow-engine.md`
- `docs/adr/008-event-driven-processing.md`
- `docs/adr/009-queue-strategy.md`
- `docs/adr/010-integration-abstraction.md`
- `docs/adr/011-authentication-rbac.md`
- `docs/adr/012-observability.md`

Engineering agent guidance

- `AGENTS.md`

Delivery

- `backlog/technical-backlog.md`

Source of truth

When implementation conflicts with these documents, do not silently improvise.

Create or update an ADR, then update the affected SDD and backlog.

Current scope boundary

V0.1 explicitly excludes:

- full accounting;
- payments processing;
- full project management;
- mobile applications;
- social media management;
- custom/fine-tuned LLMs;

- autonomous high-risk financial actions;
- a replacement for every contractor-specific system.

Official technology references

- Nuxt: https://nuxt.com/
- NestJS modules: https://docs.nestjs.com/modules
- NestJS queues / BullMQ: https://docs.nestjs.com/techniques/queues
- PostgreSQL RLS: https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- PostgreSQL policies: https://www.postgresql.org/docs/current/sql-createpolicy.html
- Turborepo: https://turborepo.com/
- Prisma: https://www.prisma.io/

AGENTS.md

AGENTS.md — Vautrin OS Engineering Contract

This file is the default instruction set for AI coding agents working in the Vautrin OS repository.

1. Mission

Build Vautrin OS as a secure, multi-tenant, modular SaaS platform for local businesses.

The first vertical is remodeling/home services.

Do not optimize for feature volume. Optimize for:

• correctness;

• tenant isolation;

• maintainability;

• testability;

• observable behavior;

• small, reviewable changes.

2. Architecture rules

1. The repository is a pnpm workspace managed by Turborepo.
2. The frontend is Nuxt 4 + Vue 3 + TypeScript.
3. The backend is NestJS + TypeScript.
4. The backend is a modular monolith.
5. `apps/api` is the HTTP/API application.
6. `apps/worker` handles asynchronous jobs and workflow execution.
7. PostgreSQL is the source of truth.
8. Redis/BullMQ is for asynchronous work, not durable business state.
9. pgvector is used for semantic knowledge retrieval.
10. Integrations must be accessed through provider interfaces.
11. LLM providers must be hidden behind an internal provider abstraction.
12. Domain logic must not depend directly on HTTP controllers.
13. Domain logic must not depend directly on an LLM vendor.
14. AI agents may act only through explicitly authorized tools.
15. Tenant-scoped data must always carry tenant context.
16. Do not introduce microservices unless an ADR justifies extraction.

3. Multi-tenancy rules

• Every tenant-scoped table has `tenant_id`.

• Every repository query must be tenant-aware.

• PostgreSQL RLS is required for tenant-scoped data.

• Never accept a tenant ID from a client as an authorization decision.

• Derive the active tenant from authenticated membership/context.

• Cross-tenant access must have automated tests.

• Background jobs must carry tenant context explicitly.

• Logs must contain tenant context where applicable.

4. Database rules

• Use migrations; never use `db push` in production.

• Do not mutate production schema manually.

• Add indexes based on actual access patterns.

• Use UUID/UUIDv7-style identifiers rather than sequential public IDs.

• Keep structured business data relational.

• Use JSONB only for genuinely flexible configuration.

• Use pgvector for unstructured semantic retrieval, not as a replacement for relational queries.

• Every migration must be reviewable and reversible where practical.

5. NestJS rules

• Use modules to encapsulate business capabilities.

• Controllers are thin.

• Application services orchestrate use cases.

• Domain logic belongs outside controllers.

• Infrastructure adapters implement interfaces.

• Avoid circular dependencies.

• Prefer explicit dependency injection.

6. AI rules

• Never give an LLM direct database access.

• Never allow arbitrary tool execution.

• Every tool has a typed input/output contract.

• Every sensitive tool has authorization and validation.

• High-impact actions may require human approval.

• Prompts are versioned artifacts.

• Model/provider configuration is tenant-aware.

• Store AI execution metadata required for debugging and cost analysis.

• Never fabricate business facts when the source of truth is unavailable.

• When uncertain, the agent should ask, escalate, or decline rather than invent.

7. Workflow rules

• Workflows are event-driven.

• Workflow definitions must be data/configuration, not scattered business conditionals.

• Workflow execution must be idempotent.

• Retries must be bounded and observable.

• Long-running work belongs in BullMQ workers.

• A workflow must define failure behavior.

• Actions that send external messages should have idempotency protection.

8. API rules

• Validate external input at the boundary.

• Return stable DTOs/contracts.

• Use consistent error responses.

• Do not leak internal exceptions.

• Document public API behavior with OpenAPI.

• Pagination is mandatory for list endpoints expected to grow.

9. Testing rules

Every feature should include the smallest meaningful test set:

• unit tests for domain/application behavior;

• integration tests for persistence/infrastructure;

• tenant-isolation tests where data access is involved;

• E2E tests for critical user workflows.

AI features additionally require:

• deterministic mocked tool tests;

• prompt/behavior evaluation cases;

• hallucination/grounding checks;

• failure and escalation tests.

10. Security rules

• Secrets never enter source control.

• Use environment/secret-manager configuration.

• Do not log tokens, passwords, raw credentials, or sensitive customer data unnecessarily.

• Audit important state-changing actions.

• Principle of least privilege applies to users, agents, integrations, and infrastructure.

11. Code quality

• TypeScript strict mode.

• Avoid `any`; justify exceptions.

• Prefer explicit types at system boundaries.

• Avoid premature abstractions.

• Prefer small cohesive modules.

• Do not duplicate domain rules between frontend and backend.

• Use shared contracts for API boundaries where appropriate.

12. Change protocol for AI agents

Before changing architecture:

1. Read the relevant SDD.

2. Read relevant ADRs.

3. Identify impacted modules.

4. Update the implementation plan.

5. Implement the smallest coherent change.

6. Add tests.

7. Run affected checks.

8. Update documentation when behavior/architecture changes.

If a task contradicts an ADR, do not silently override it. Propose an ADR change.

13. Definition of Done

A task is not complete when the code merely runs.

It is complete when:

• acceptance criteria are met;

• tests pass;

• tenant isolation is verified where relevant;

• migrations are included;

• observability is present for operational behavior;

• API contracts are updated;

• documentation is updated when necessary;

• CI passes.

14. What not to build in V0.1

Do not introduce:

• microservice infrastructure;

• Kubernetes;

• custom LLM training;

• fine-tuning pipelines;

• a complete accounting platform;

• a complete project-management replacement;

• a native mobile app;

• a marketplace;

• autonomous financial decisions.

Build the revenue-engine vertical slice first.

00-overview.md

SDD-00 — Vautrin OS Overview

1. Purpose

Vautrin OS is a multi-tenant SaaS platform that connects customer acquisition, lead management, AI-assisted communication, sales workflows, appointments, estimates, and later operational workflows into one business operating layer.

The MVP is intentionally optimized for remodeling and home-service companies in the United States while keeping the platform core vertical-agnostic.

2. Product thesis

The platform should not be sold as "AI automation".

It should eliminate operational waste around:

• missed calls;

• slow lead response;

• inconsistent qualification;

• forgotten follow-ups;

• untracked estimates;

• disconnected communication channels.

3. MVP promise

> Capture, qualify, follow up with, and book local-business leads with an AI-assisted revenue workflow.

4. MVP lifecycle

```
Inbound lead
  -> Contact identification
  -> Lead creation/update
  -> AI reception
  -> Qualification
  -> Follow-up
  -> Appointment
  -> Estimate
  -> Estimate follow-up
```

5. Architectural goals

G1 — Multi-tenancy

Adding a new business must not require a new application stack.

G2 — Vertical extensibility

New verticals should be introduced through templates/configuration wherever possible.

G3 — Provider independence

LLM, messaging, telephony, and calendar providers must be replaceable.

G4 — Operational reliability

External calls must be asynchronous where appropriate and retryable.

G5 — AI safety

Agents can only act through explicit, authorized tools.

G6 — Developer velocity

The architecture must be practical for a small engineering team assisted by coding agents.

6. Non-goals

V0.1 is not:

• an accounting system;

• a payment processor;

• a complete project-management suite;

• a general-purpose CRM replacement;

• an autonomous financial operator;

• a custom LLM platform.

7. Success criteria

The MVP is successful when a real remodeling business can:

1. configure its business profile;

2. manage contacts and leads;

3. receive inbound communication through an integrated channel;

4. have an AI agent qualify a lead;

5. execute a follow-up workflow;

6. book an appointment;

7. track an estimate;

8. automatically follow up on the estimate;

9. observe the workflow in the dashboard;

10. operate without cross-tenant data exposure.

8. Technology baseline

• Nuxt 4 / Vue 3

• NestJS

• TypeScript

• PostgreSQL

• pgvector

• Redis

• BullMQ

- pnpm
- Turborepo
- GitHub Actions
- Docker

01-requirements.md

SDD-01 — Requirements

1. Functional requirements

FR-001 Tenant management

The system shall support multiple independent businesses.

FR-002 User membership

A user may belong to one or more tenants subject to authorization rules.

FR-003 Business configuration

Each tenant shall configure:

• business name;

• locations;

• service areas;

• services;

• business hours;

• communication preferences;

• qualification rules;

• booking rules;

• AI configuration.

FR-004 Contacts

Users shall create, view, update, search, and paginate contacts.

FR-005 Leads

Users shall create and manage leads with lifecycle status, source,

service, assigned user, and custom fields.

FR-006 Conversations

The system shall represent customer conversations independently of channel.

FR-007 Messages

Messages shall record direction, channel, status, timestamps, and provider metadata.

FR-008 AI reception

The AI receptionist shall:

• understand basic customer intent;

• answer grounded business questions;

• collect lead information;

• create/update leads;

• qualify leads;

• attempt appointment booking;

• escalate when required.

FR-009 Appointments

The system shall expose availability and create/cancel appointments through a provider abstraction.

FR-010 Estimates

The system shall track estimate creation, sending, status, and follow-up state.

FR-011 Workflows

The system shall execute event-triggered workflows with conditions, waits, actions, retries, and failure handling.

FR-012 Knowledge

The system shall support tenant-specific unstructured knowledge and semantic retrieval.

FR-013 Audit

Important state changes and external actions shall be auditable.

2. Non-functional requirements

NFR-001 Tenant isolation

No tenant may access another tenant's data.

NFR-002 Reliability

External side effects should be retryable and idempotent where practical.

NFR-003 Performance

Normal dashboard/API operations should be responsive enough for interactive use;

long-running operations must not block HTTP requests.

NFR-004 Security

Secrets, credentials, and sensitive data must be protected.

NFR-005 Observability

The platform shall provide structured logs and correlation identifiers.

NFR-006 Testability

Domain behavior and critical workflows must be testable without real external providers.

NFR-007 Extensibility

Adding a new vertical should primarily require configuration/templates.

3. Initial user roles

• Owner

• Admin

• Manager

• Staff

• Read-only

Agent execution is not treated as a human role.

4. Initial lead statuses

```
NEW
CONTACTED
QUALIFIED
APPOINTMENT_BOOKED
ESTIMATE_PENDING
ESTIMATE_SENT
FOLLOW_UP
WON
LOST
```

The status model must remain configurable enough to support future verticals.

5. Critical user stories

US-001

As a contractor owner, I can see all new leads in one pipeline.

US-002

As a contractor owner, I can configure services and service areas.

US-003

As a customer, I can receive a fast response after contacting the business.

US-004

As a business, I can have AI collect qualification information.

US-005

As a business, I can have AI offer available appointment times.

US-006

As a business, I can automatically follow up with leads who do not respond.

US-007

As a business, I can track estimates awaiting customer response.

US-008

As an administrator, I can inspect AI actions and workflow executions.

6. Acceptance baseline

A release candidate must demonstrate the complete critical path:

```
Inbound event
 -> tenant resolution
 -> contact
```

```
-> lead
-> conversation
-> AI/tool execution
-> follow-up
-> appointment
-> estimate
-> estimate follow-up
-> audit
```

02-architecture.md

SDD-02 — Architecture

1. Architecture style

Vautrin OS uses a **modular monolith** for the MVP.

This gives us:

• one deployable API;

• strong module boundaries;

• low operational overhead;

• fast development;

• a clear extraction path for future services.

The asynchronous worker is a separate deployable application from the beginning.

2. Repository architecture

```
vautrin-os/
■■■ apps/
■   ■■■ web/
■   ■■■ api/
■   ■■■ worker/
■■■ packages/
■   ■■■ config/
■   ■■■ database/
■   ■■■ domain/
■   ■■■ contracts/
■   ■■■ auth/
■   ■■■ ai/
■   ■■■ workflows/
■   ■■■ integrations/
■   ■■■ observability/
■   ■■■ testing/
■■■ templates/
■   ■■■ remodeling/
■   ■■■ hvac/
■   ■■■ roofing/
■   ■■■ generic-local-business/
■■■ docs/
■■■ infra/
■■■ .github/
```

3. Applications

Web

Nuxt 4 + Vue 3.

API

NestJS HTTP application.

Worker

NestJS process responsible for asynchronous processing.

4. Backend module map

```
AuthModule
TenantModule
BusinessModule
ContactModule
LeadModule
ConversationModule
AppointmentModule
EstimateModule
WorkflowModule
AgentModule
KnowledgeModule
IntegrationModule
AnalyticsModule
AuditModule
```

5. Dependency direction

```
Presentation
    ↓
Application
    ↓
Domain
    ↑
Infrastructure
```

Infrastructure implements interfaces defined by the application/domain boundary.

Domain must not import Prisma, Redis, HTTP clients, or LLM SDKs directly.

6. API boundary

The API exposes DTOs/contracts.

Frontend must not depend on backend internal domain entities.

Shared contracts are allowed for:

• request schemas;

• response DTOs;

• event schemas;

• public enums.

7. Worker boundary

The API publishes jobs/events.

The worker consumes them.

Example:

```
POST /leads
  -> database transaction
  -> publish lead.created
  -> enqueue follow-up/AI work if required
  -> HTTP response
```

8. Scalability path

MVP:

```
Nuxt
```

```
Nest API
Nest Worker
Postgres
Redis
```

Future extraction candidates:

```
Voice Service
AI Execution Service
Workflow Worker Pool
Analytics Pipeline
Integration Gateway
```

Extraction requires an ADR and measurable justification.

9. Deployment topology

```
Internet
  |
Load Balancer / Edge
  |
Nuxt Web
  |
Nest API
  |
  +---- PostgreSQL
  +---- Redis
  |
Worker
  |
External Providers
```

10. Technology principles

• Prefer boring infrastructure.

• Keep business state in PostgreSQL.

• Keep queues disposable/replayable.

• Keep external providers behind adapters.

• Keep AI behind interfaces.

• Keep tenant isolation explicit at every layer.

03-domain-model.md

SDD-03 — Domain Model

1. Core entities

Tenant

The security and billing boundary for a business account.

User

A human identity.

TenantUser

Membership connecting a user to a tenant and role.

BusinessProfile

Business-specific operational configuration.

Location

Physical/service location associated with a tenant.

Contact

A person or organization interacting with the business.

Lead

A sales opportunity associated with a contact.

Conversation

A channel-independent interaction thread.

Message

An inbound/outbound communication item.

Call

A telephony interaction and its metadata/transcript reference.

Appointment

A scheduled customer interaction.

Estimate

A sales estimate and its lifecycle.

Workflow

A declarative automation definition.

WorkflowRun

One execution instance of a workflow.

WorkflowStepRun

Execution state for an individual workflow step.

Agent

An AI capability definition.

AgentRun

One execution of an AI agent.

ToolExecution

One invocation of an agent tool.

KnowledgeDocument

A tenant-owned source document.

KnowledgeChunk

An embedded semantic chunk of a knowledge document.

Integration

Connection metadata for an external provider.

Event

A durable business event emitted by the platform.

AuditLog

A security/operations record of important actions.

2. Relationship model

```
Tenant
 ■■■ TenantUser ■■ User
 ■■■ BusinessProfile
 ■■■ Location
 ■■■ Contact
 ■    ■■■ Lead
 ■         ■■■ Conversation
 ■         ■    ■■■ Message
 ■         ■■■ Call
 ■         ■■■ Appointment
 ■         ■■■ Estimate
 ■■■ Workflow
 ■    ■■■ WorkflowRun
 ■         ■■■ WorkflowStepRun
 ■■■ Agent
 ■    ■■■ AgentRun
 ■         ■■■ ToolExecution
 ■■■ KnowledgeDocument
 ■    ■■■ KnowledgeChunk
 ■■■ Integration
 ■■■ AuditLog
```

3. Tenant-scoped rule

All entities except global catalog/configuration entities are tenant-scoped.

Every tenant-scoped entity must contain `tenant_id`.

4. Lifecycle state

Lead lifecycle is a business concept, not an AI concept.

AI may recommend or perform transitions through tools, but the domain owns the valid state transitions.

5. Custom fields

Vertical-specific attributes are represented using:

- `CustomFieldDefinition`
- `CustomFieldValue`

Do not add vertical-specific columns to core entities unless the field becomes a universal concept.

6. Vertical templates

Templates provide:

- services;
- custom fields;
- qualification questions;
- default workflows;
- prompt/context configuration;
- dashboard configuration.

Templates must not fork the core domain model.

04-database.md

SDD-04 — Database Specification

1. PostgreSQL

PostgreSQL is the source of truth for durable business state.

2. Required extension

```
CREATE EXTENSION IF NOT EXISTS vector;
```

The exact embedding dimensions are provider/model configuration and must not be hardcoded into unrelated domain code.

3. Initial tables

```
tenants
users
tenant_users
business_profiles
locations

contacts
leads
lead_sources

conversations
messages
calls

appointments
estimates

workflows
workflow_runs
workflow_step_runs

agents
agent_runs
tool_executions

knowledge_documents
knowledge_chunks

integrations
events
audit_logs

custom_field_definitions
custom_field_values
```

4. Common columns

Tenant-scoped tables should normally contain:

```
id
tenant_id
created_at
updated_at
```

Where useful:

```
deleted_at
```

```
created_by
updated_by
version
```

5. IDs

Use UUID/UUIDv7-style IDs.

IDs exposed through APIs must not reveal tenant sequencing.

6. Indexing baseline

Every tenant-scoped high-volume table should have an index beginning with

`tenant_id` where queries commonly filter by tenant.

Examples:

```
leads(tenant_id, status)
leads(tenant_id, created_at)
contacts(tenant_id, phone)
conversations(tenant_id, updated_at)
appointments(tenant_id, starts_at)
workflow_runs(tenant_id, status)
audit_logs(tenant_id, created_at)
```

Exact indexes must be validated against query plans.

7. JSONB

Use JSONB for:

• flexible integration metadata;

• workflow configuration;

• provider-specific metadata;

• tenant configuration that is intentionally schema-flexible.

Do not use JSONB to avoid modeling stable business concepts.

8. Vector storage

`knowledge_chunks` should contain:

```
id
tenant_id
document_id
content
embedding
metadata
created_at
```

Metadata can include:

• document type;

• service;

• location;

• source;

• version.

9. Transactions

Use transactions for state transitions that must remain atomic.

Examples:

• create lead + initial event;

• create appointment + update lead status;

• workflow state + event emission.

For cross-system side effects, use an outbox-style pattern when required rather than pretending the external provider participates in the database transaction.

10. Migrations

Production uses:

```
prisma migrate deploy
```

Never use:

```
prisma db push
```

against production.

11. Backup/recovery

The production database must have:

• automated backups;

• restore procedure;

• tested recovery process;

• documented retention policy.

Exact infrastructure values are environment decisions, not hardcoded in the domain.

05-multitenancy.md

SDD-05 — Multi-Tenancy

1. Model

Use a pooled PostgreSQL multi-tenant model.

All tenant data resides in shared database infrastructure with logical isolation.

2. Tenant context

The active tenant is derived from:

1. authenticated identity;

2. tenant membership;

3. explicit server-side tenant selection where the user belongs to multiple tenants.

A client-provided arbitrary tenant ID is never sufficient authorization.

3. Request flow

```
HTTP request
  ↓
Authentication
  ↓
Tenant membership resolution
  ↓
TenantContext
  ↓
Authorization
  ↓
Application service
  ↓
Repository
  ↓
PostgreSQL
```

4. RLS

All tenant-scoped tables should have PostgreSQL Row Level Security enabled.

The application must set a database session/transaction tenant context safely

before executing tenant-scoped operations.

Example conceptual policy:

```
CREATE POLICY tenant_isolation
ON leads
USING (tenant_id = current_setting('app.tenant_id')::uuid)
WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
```

Implementation must account for connection pooling and transaction boundaries.

5. Background jobs

Every job must carry enough context to reconstruct:

• tenant ID;

• actor/system identity where applicable;

• correlation ID;

• idempotency key where relevant.

A worker must never infer tenant context from mutable global process state.

6. Tenant isolation tests

Required tests include:

• SELECT isolation;

• INSERT isolation;

• UPDATE isolation;

• DELETE isolation;

• vector/knowledge isolation;

• conversation isolation;

• workflow isolation;

• background job isolation.

7. Noisy neighbor

At MVP scale, shared infrastructure is acceptable.

As customer volume grows, monitor:

• database CPU;

• connection usage;

• slow queries;

• queue depth;

• worker saturation;

• tenant-specific request volume.

Future isolation options may include:

• rate limits;

• queue partitioning;

• read replicas;

• dedicated infrastructure for enterprise tenants.

8. Tenant deletion

Tenant deletion must be an explicit lifecycle operation with:

• authorization;

• audit;

• data retention policy;

• asynchronous cleanup;

• provider credential revocation.

06-ai-system.md

SDD-06 — AI System

1. AI is an application capability

AI is not the source of truth.

The source of truth is:

• PostgreSQL;

• configured business rules;

• approved knowledge;

• external systems through verified tools.

2. Provider abstraction

```
interface LLMProvider {
  generate(request: LLMRequest): Promise<LLMResponse>;
  stream(request: LLMRequest): AsyncIterable<LLMChunk>;
  embed(input: string[]): Promise<Embedding[]>;
}
```

Providers may include:

• OpenAI;

• Anthropic;

• xAI;

• AWS Bedrock;

• future local providers.

3. Agent abstraction

An agent contains:

```
instructions
model configuration
allowed tools
business context strategy
knowledge strategy
guardrails
handoff policy
```

4. Agent execution

```
User event
  ↓
Load tenant
  ↓
Load business configuration
  ↓
Load conversation/contact/lead state
  ↓
Retrieve relevant knowledge
  ↓
Build model context
  ↓
LLM
  ↓
```

```
Tool call?
  ■■■ No → response validation → output
  ■■■ Yes
       ↓
     authorize
       ↓
     validate
       ↓
     execute
       ↓
     audit
       ↓
     continue
```

5. Tool security

Each tool declares:

• name;

• description;

• input schema;

• output schema;

• permission;

• side-effect level;

• audit requirement.

Example:

```
create_appointment
permission: appointment.write
side_effect: external
approval: no/conditional
audit: required
```

6. Business context

Structured context should be loaded from relational data.

Semantic context should be retrieved from pgvector.

Do not use RAG for:

• current appointment availability;

• lead status;

• exact customer identity;

• transactional state.

7. Grounding

The agent must distinguish:

• known facts;

• retrieved knowledge;

• user-provided facts;

• uncertain information.

When a business fact is unavailable, the agent must not invent it.

8. Human handoff

The agent should hand off when:

• requested action is outside permissions;

• confidence/grounding is insufficient;

• customer requests a human;

• business policy requires human review;

• an integration fails repeatedly.

9. Prompt versioning

Prompts are versioned artifacts.

Every production agent run should record:

• agent version;

• prompt version;

• model;

• provider;

• tool calls;

• latency;

• token usage/cost where available;

• final status.

10. AI evaluation

Maintain a regression dataset containing:

• normal customer requests;

• ambiguous requests;

• unsupported requests;

• adversarial requests;

• tenant-specific business facts;

• tool execution cases;

• escalation cases.

The dataset is part of the repository and must not contain real customer PII.

07-workflow-engine.md

SDD-07 — Workflow Engine

1. Purpose

The workflow engine turns business events into reliable, configurable actions.

It is the foundation for:

• missed-call recovery;

• lead follow-up;

• estimate follow-up;

• customer re-engagement;

• review automation;

• future operations workflows.

2. Workflow model

```
Workflow
 ■■■ Trigger
 ■■■ Conditions
 ■■■ Steps
 ■■■ Retry policy
 ■■■ Failure policy
 ■■■ Activation state
```

3. Trigger

Examples:

```
call.missed
lead.created
lead.updated
message.received
appointment.completed
estimate.sent
customer.inactive
```

4. Step types

V0.1:

```
wait
condition
send_message
create_lead
update_lead
create_appointment
call_agent
create_task
human_approval
end
```

5. Execution

```
Event
 ↓
Find active workflows
```

```
 ↓
Create WorkflowRun
 ↓
Enqueue step
 ↓
Worker executes
 ↓
Persist step result
 ↓
Next step
```

6. Idempotency

Every side-effecting workflow step should have an idempotency key.

Example:

```
tenant_id
workflow_run_id
step_id
target_id
```

The same external SMS must not be sent twice because a worker retried.

7. Retry policy

Retries should define:

• maximum attempts;

• exponential/backoff strategy;

• retryable errors;

• terminal errors.

8. Failure policy

Each workflow must specify what happens when a step fails:

• retry;

• skip;

• stop;

• escalate;

• human handoff.

9. Scheduling

Delayed steps are persisted as queue jobs.

Do not rely on in-memory timers.

10. Workflow versioning

Published workflow versions are immutable.

A new configuration creates a new version.

Existing workflow runs continue using the version with which they started.

11. Example — missed call recovery

```
{
  "trigger": "call.missed",
  "steps": [
    {"type": "wait", "duration": "30s"},
    {"type": "call_agent", "agent": "receptionist"},
    {"type": "send_message", "channel": "sms"},
    {"type": "wait", "duration": "24h"},
    {"type": "condition", "expression": "lead.replied == false"},
    {"type": "call_agent", "agent": "follow_up"}
  ]
}
```

The exact schema is an implementation task; the example is conceptual.

08-integrations.md

SDD-08 — Integration Architecture

1. Principle

External providers are adapters, not domain dependencies.

The domain should ask:

```
createAppointment()
sendMessage()
getAvailability()
```

not:

```
createGoogleCalendarEvent()
sendTwilioMessage()
```

2. Provider interfaces

Messaging

```
interface MessagingProvider {
  sendMessage(input: SendMessageInput): Promise<SendMessageResult>;
}
```

Calendar

```
interface CalendarProvider {
  getAvailability(input: AvailabilityInput): Promise<Availability[]>;
  createAppointment(input: CreateAppointmentInput): Promise<AppointmentResult>;
  cancelAppointment(input: CancelAppointmentInput): Promise<void>;
}
```

Voice

```
interface VoiceProvider {
  handleCall(input: IncomingCall): Promise<CallSession>;
  transfer(input: TransferCallInput): Promise<void>;
}
```

3. Integration lifecycle

```
Disconnected
   ↓
Connected
   ↓
Healthy
   ↓
Degraded
   ↓
Disconnected
```

Health state is observable.

4. Credentials

Provider credentials are:

• encrypted at rest where infrastructure supports it;

• never returned to the frontend;

• never placed in logs;

• scoped to tenant/integration;

• revocable.

5. Webhooks

Webhook handlers must:

• authenticate/verify provider signatures;

• resolve tenant;

• validate payload;

• deduplicate events;

• persist raw metadata only when appropriate;

• emit internal events.

6. Initial integrations

The architecture is ready for:

• telephony;

• SMS;

• email;

• Google Calendar;

• Microsoft Calendar;

• WhatsApp;

• CRM integrations.

V0.1 should implement only the minimum providers needed for the first vertical slice.

7. Provider portability

Switching a provider should require changing an adapter/configuration,

not domain workflows.

09-security.md

SDD-09 — Security Specification

1. Threat model

The platform handles:

• customer contact information;

• business information;

• conversations;

• appointments;

• estimates;

• integration credentials;

• AI-generated actions.

Primary threats:

• cross-tenant access;

• credential leakage;

• unauthorized AI actions;

• webhook spoofing;

• prompt injection;

• data leakage through logs;

• insecure background jobs.

2. Authentication

Use short-lived access credentials and a secure refresh/session mechanism.

Authentication identifies the user.

Authorization determines what the user can do.

3. Authorization

Permissions are scoped to:

• tenant;

• role;

• capability.

Example:

```
lead.read
lead.write
appointment.read
appointment.write
estimate.read
estimate.write
workflow.manage
integration.manage
audit.read
```

4. AI authorization

AI tools have their own permissions.

A user may have:

```
estimate.write
```

without granting an AI agent:

```
estimate.approve
```

5. Prompt injection

Retrieved/customer-provided text is untrusted input.

The system must not treat arbitrary customer content as system policy.

Business rules and tool permissions live outside the model's generated text.

6. Sensitive actions

Potentially high-impact actions require:

• explicit authorization;

• validation;

• audit;

• optional human approval.

Examples:

• changing estimate totals;

• refunds;

• deleting customer data;

• changing business rules.

7. Secrets

Secrets must be injected through environment/secret-management infrastructure.

Never commit:

• API keys;

• OAuth client secrets;

• database passwords;

• JWT secrets;

• provider tokens.

8. Logging

Do not log:

• raw credentials;

• access tokens;

• passwords;

• unnecessary full message bodies;

• unnecessary customer PII.

Use IDs and metadata wherever possible.

9. Data retention

Retention policies must be defined before production scale.

The system must support future deletion/anonymization workflows.

10. Audit

Audit state-changing actions involving:

• authentication;

• authorization;

• AI tools;

• integrations;

• workflow activation;

• customer/lead mutations;

• high-impact operations.

10-observability.md

SDD-10 — Observability

1. Goals

The platform must answer:

• what happened?

• for which tenant?

• for which entity?

• which workflow/agent caused it?

• how long did it take?

• did an external provider fail?

• how much did AI execution cost?

2. Structured logging

Every request/job should carry:

```
request_id
correlation_id
tenant_id
actor_id
operation
status
duration_ms
```

3. AI telemetry

Record where available:

```
agent_run_id
agent_version
prompt_version
provider
model
input_tokens
output_tokens
latency_ms
tool_count
failure_reason
handoff
```

4. Workflow telemetry

```
workflow_run_id
workflow_version
tenant_id
trigger_event
step
attempt
status
duration
error
```

5. Metrics

Initial metrics:

```
http_requests_total
http_request_duration_ms
queue_depth
queue_failures
workflow_runs_total
workflow_failures_total
ai_runs_total
ai_failures_total
tool_executions_total
integration_errors_total
```

Business metrics:

```
leads_created
leads_qualified
appointments_booked
estimates_sent
estimate_followups
missed_calls
recovered_leads
```

6. Correlation

A customer event should be traceable:

```
call
 ↓
webhook
 ↓
event
 ↓
workflow
 ↓
agent
 ↓
tool
 ↓
SMS
```

using a shared correlation ID.

7. Alerts

Production alerting should eventually cover:

• API error rate;

• queue backlog;

• worker failures;

• database saturation;

• integration outage;

• unusual AI error/cost spikes.

11-testing.md

SDD-11 — Testing Strategy

1. Test pyramid

```
       E2E
      /    Integration
   /        Unit / Domain
```

2. Unit tests

Test:

• domain state transitions;

• qualification rules;

• workflow conditions;

• permission logic;

• tool input validation.

3. Integration tests

Test:

• repositories;

• PostgreSQL;

• RLS;

• Redis/BullMQ;

• provider adapters with mocks/test servers.

4. Tenant isolation tests

Every security-sensitive repository should include cross-tenant tests.

Minimum:

```
Tenant A cannot read Tenant B
Tenant A cannot write Tenant B
Tenant A cannot delete Tenant B
```

5. E2E tests

Critical path:

```
Create tenant
 -> configure business
 -> create lead
 -> receive inbound event
 -> AI qualification
 -> follow-up
 -> book appointment
 -> create estimate
 -> estimate follow-up
```

6. AI evaluation

AI tests are behavioral regression tests, not exact string tests.

Evaluate:

• correctness;

• grounding;

• tool selection;

• parameter correctness;

• refusal/escalation;

• hallucination resistance;

• business-rule compliance.

7. External providers

Never make normal CI depend on real paid providers.

Use:

• mocks;

• contract tests;

• sandbox environments where available.

8. Test data

Synthetic data only.

Never commit real customer PII.

9. CI gates

A pull request must pass:

• lint;

• typecheck;

• unit;

• integration;

• build;

• affected E2E tests;

• security/secret checks.

12-deployment.md

SDD-12 — Deployment and CI/CD

1. Environments

```
local
staging
production
```

2. Local development

Docker Compose should provide:

• PostgreSQL + pgvector;

• Redis;

• optional Mailpit;

• optional local provider mocks.

3. CI pipeline

```
Pull Request
  ↓
Checkout
  ↓
Install pnpm dependencies
  ↓
Turborepo affected lint/typecheck/test
  ↓
Build affected applications
  ↓
Integration tests
  ↓
E2E tests
  ↓
Security checks
```

4. Merge

`main` must remain deployable.

5. Staging deployment

On successful merge:

• build container images;

• publish immutable image tags;

• deploy API;

• deploy worker;

• deploy web;

• run database migrations;

• run smoke tests.

6. Production deployment

Production should require:

• successful staging;

• migration safety check;

• explicit approval if the environment requires it;

• deployment;

• health checks;

• smoke tests;

• rollback path.

7. Database migration policy

Migrations are deployed separately from application startup where practical.

Application startup must not race multiple instances attempting arbitrary schema changes.

8. Container strategy

Use one Dockerfile per deployable application where practical:

• web;

• api;

• worker.

Shared packages are compiled into the relevant application image.

9. Image tagging

Use immutable tags based on commit SHA.

Human-readable release tags may be added, but deployments must be reproducible.

10. Rollback

Application rollback must be possible independently from database rollback where feasible.

Destructive database migrations require expand/contract planning.

11. Monorepo optimization

Turborepo should determine affected tasks using the repository dependency graph.

Do not run expensive E2E/build steps for unrelated changes unless the task graph
or safety policy requires them.

12. Production readiness

Before the first real customer:

• backups verified;

• restore procedure tested;

- secrets configured;
- tenant isolation tests green;
- audit/logging verified;
- monitoring configured;
- provider failure paths tested.

001-monorepo-strategy.md

ADR-001 — Monorepo Strategy

• Status: Accepted

• Date: 2026-09-19

Context

Vautrin OS contains a web application, API, worker processes, shared contracts, domain libraries, AI infrastructure, integration adapters, and vertical templates. Separate repositories would make synchronized changes and AI-assisted development more expensive.

Decision

Use a single repository managed by:

• pnpm workspaces;

• Turborepo.

NestJS remains the backend framework but does not own the repository-level monorepo strategy.

Repository layout

```
apps/
packages/
templates/
docs/
infra/
```

Consequences

Positive:

• shared TypeScript packages;

• unified CI;

• dependency graph awareness;

• atomic changes;

• consistent tooling.

Negative:

• repository complexity grows;

• CI configuration needs affected-task discipline;

• package boundaries must be enforced.

Rejected alternatives

Separate repositories

Rejected for MVP because they increase coordination overhead.

Nest CLI monorepo as the repository strategy

Rejected because the repository contains Nuxt, infrastructure, templates, and non-Nest packages in addition to Nest applications.

002-modular-monolith.md

ADR-002 — Modular Monolith

• Status: Accepted

• Date: 2026-09-19

Context

The MVP needs strong domain boundaries but must remain inexpensive to develop and operate.

Decision

Use a NestJS modular monolith for the API and a separate NestJS worker process.

Modules encapsulate:

• auth;

• tenant;

• business;

• CRM;

• conversations;

• appointments;

• estimates;

• workflows;

• AI;

• knowledge;

• integrations;

• analytics;

• audit.

Consequences

Positive:

• fast development;

• simple deployment;

• shared transactions;

• clear module ownership.

Negative:

• a single API can become large;

• module boundaries must be actively maintained.

Extraction rule

A module may become a separate service only when:

- scale requires it;
- deployment independence has material value;
- failure isolation is required;
- ownership/team boundaries justify it.

An ADR is required before extraction.

003-multi-tenancy.md

ADR-003 — Pooled Multi-Tenancy

• Status: Accepted

• Date: 2026-09-19

Context

The product must support many local businesses without provisioning an entire application stack per customer.

Decision

Use a pooled PostgreSQL model with logical tenant isolation.

Every tenant-scoped row contains `tenant_id`.

Tenant isolation is enforced at:

1. application context;
2. repository/query boundaries;
3. PostgreSQL RLS;
4. automated tests.

Consequences

Positive:

• simple onboarding;

• low infrastructure overhead;

• efficient resource utilization;

• shared migrations.

Negative:

• stronger isolation engineering is required;

• noisy-neighbor behavior must be monitored.

Future

Enterprise tenants may receive dedicated databases/stacks if commercial or security requirements justify it.

004-postgresql-pgvector.md

ADR-004 — PostgreSQL + pgvector

• Status: Accepted

• Date: 2026-09-19

Context

The platform needs relational business state and semantic retrieval for tenant knowledge.

Decision

Use PostgreSQL as the source of truth and pgvector for embeddings.

Structured data stays relational.

Unstructured knowledge is chunked and embedded in PostgreSQL.

Consequences

Positive:

• one primary database;

• transactional consistency;

• simpler operations;

• tenant isolation can apply to vector data;

• lower MVP complexity.

Negative:

• vector workload must be monitored;

• very large-scale retrieval may eventually require specialized infrastructure.

Migration path

If vector workload outgrows PostgreSQL, introduce a vector provider adapter without changing the domain-level knowledge interface.

005-tenant-isolation-rls.md

ADR-005 — PostgreSQL RLS for Tenant Isolation

• Status: Accepted

• Date: 2026-09-19

Context

Application-only tenant filtering is vulnerable to accidental omissions.

Decision

Use PostgreSQL Row Level Security for tenant-scoped tables in addition to application-level tenant context.

The active tenant is passed through a safe transaction/session context.

Consequences

Positive:

• defense in depth;

• database-level enforcement;

• protection against accidental query omissions.

Negative:

• connection pooling and transaction boundaries require careful implementation;

• test setup is more involved;

• privileged database roles require explicit handling.

Rule

RLS is not a replacement for application authorization. Both are required.

006-ai-provider-abstraction.md

ADR-006 — AI Provider Abstraction

• Status: Accepted

• Date: 2026-09-19

Context

Model providers and pricing/performance change quickly. The business must not be coupled to one vendor.

Decision

Define internal interfaces for:

• text generation;

• streaming;

• embeddings.

Implement providers as adapters.

Consequences

Positive:

• provider portability;

• easier testing;

• controlled cost optimization;

• future model routing.

Negative:

• abstraction has to cover meaningful common capabilities;

• provider-specific features may require optional capabilities.

Rule

Do not expose provider SDK types through the domain layer.

007-workflow-engine.md

ADR-007 — Declarative Workflow Engine

• Status: Accepted

• Date: 2026-09-19

Context

The business requires many automations that share common behavior:

triggers, waits, conditions, AI actions, messages, booking, and escalation.

Hardcoding each workflow would create duplication and make vertical expansion costly.

Decision

Implement a declarative workflow engine with:

• triggers;

• versioned definitions;

• generic steps;

• retries;

• idempotency;

• persisted runs;

• failure policies.

Consequences

Positive:

• reusable automation infrastructure;

• vertical templates;

• customer-specific workflows;

• easier future workflow builder.

Negative:

• workflow engine is a significant subsystem;

• execution semantics must be tested carefully.

MVP constraint

Only implement the step types required by the first revenue workflow.

008-event-driven-processing.md

ADR-008 — Event-Driven Internal Processing

• Status: Accepted

• Date: 2026-09-19

Context

Lead, conversation, appointment, and estimate changes can trigger asynchronous behavior.

Decision

Use durable internal events as the primary trigger mechanism for workflows.

Examples:

• `lead.created`;

• `call.missed`;

• `message.received`;

• `appointment.created`;

• `estimate.sent`.

Consequences

Positive:

• loose coupling;

• replayable processing;

• natural workflow triggers;

• easier observability.

Negative:

• eventual consistency;

• event schema/versioning required;

• duplicate delivery must be handled.

Rule

Business state remains in PostgreSQL. Events trigger work; they are not the canonical replacement for business state.

009-queue-strategy.md

ADR-009 — Redis + BullMQ

• Status: Accepted

• Date: 2026-09-19

Context

The platform needs delayed jobs, retries, background AI work, webhook handling, and workflow execution.

Decision

Use Redis + BullMQ via NestJS's BullMQ integration.

Queues are processed by `apps/worker`.

Queue categories

Initial logical queues:

• `workflow`;

• `ai`;

• `notifications`;

• `webhooks`.

Consequences

Positive:

• mature Redis-backed queue model;

• delayed/retryable jobs;

• independent worker scaling;

• good NestJS integration.

Negative:

• Redis becomes operational infrastructure;

• job payloads must remain serializable and safe;

• idempotency remains an application responsibility.

010-integration-abstraction.md

ADR-010 — Provider Adapter Architecture

• Status: Accepted

• Date: 2026-09-19

Context

Messaging, voice, calendar, email, and other providers can change.

Decision

Define domain-facing interfaces and provider adapters.

Examples:

• `MessagingProvider`;

• `CalendarProvider`;

• `VoiceProvider`.

Consequences

Positive:

• provider portability;

• isolated integration tests;

• reduced domain coupling.

Negative:

• interfaces must avoid becoming lowest-common-denominator abstractions.

Rule

Provider-specific functionality belongs in adapters or explicitly scoped capability interfaces.

011-authentication-rbac.md

ADR-011 — Authentication and RBAC

• Status: Accepted

• Date: 2026-09-19

Context

The platform is multi-tenant and will contain sensitive business data.

Decision

Separate:

• authentication;

• tenant membership;

• authorization.

Initial roles:

• owner;

• admin;

• manager;

• staff;

• read-only.

Permissions are capability-based.

Consequences

Positive:

• clearer security model;

• easier future role customization;

• agent permissions can reuse capability concepts.

Negative:

• authorization must be checked consistently;

• multiple tenant memberships require explicit active-tenant handling.

012-observability.md

ADR-012 — Observability Baseline

• Status: Accepted

• Date: 2026-09-19

Context

AI workflows involve asynchronous execution and external providers. Debugging without correlation data will be expensive.

Decision

Implement structured logs and correlation IDs from the beginning.

Record:

• request/job ID;

• correlation ID;

• tenant ID;

• actor ID where applicable;

• workflow run;

• agent run;

• tool execution;

• provider/model;

• latency;

• error state.

Consequences

Positive:

• faster debugging;

• tenant-aware support;

• AI cost visibility;

• operational analytics.

Negative:

• telemetry storage has a cost;

• sensitive data must be deliberately filtered.

technical-backlog.md

Vautrin OS — Technical Backlog V0.1

Backlog rules

Priority:

• P0 = blocks MVP / security / architecture

• P1 = required for first production vertical slice

• P2 = useful MVP enhancement

• P3 = post-MVP

Status:

• TODO

• IN_PROGRESS

• BLOCKED

• DONE

Each task should be implemented as a small PR where practical.

EPIC E0 — Repository and developer experience

VOS-001 [P0] Initialize pnpm workspace

Acceptance criteria

• root package.json exists;

• pnpm-workspace.yaml exists;

• package manager version is pinned;

• Node version is pinned;

• install works from a clean checkout.

VOS-002 [P0] Configure Turborepo

Acceptance criteria

• `dev`, `build`, `lint`, `typecheck`, `test` tasks are defined;

• task dependencies are explicit;

• affected tasks can be run locally.

VOS-003 [P0] Create apps/web

Nuxt 4 + Vue 3 + TypeScript.

VOS-004 [P0] Create apps/api

NestJS application.

VOS-005 [P0] Create apps/worker

NestJS worker application.

VOS-006 [P0] Configure shared TypeScript standards

Strict mode, path conventions, linting, formatting.

VOS-007 [P0] Add AGENTS.md

Install repository engineering contract.

EPIC E1 — Local infrastructure

VOS-010 [P0] Docker Compose

PostgreSQL + pgvector + Redis.

VOS-011 [P0] Database package

Prisma configuration, migrations, client.

VOS-012 [P0] Redis/BullMQ package

Shared queue configuration.

VOS-013 [P1] Local email/mock provider

Mailpit or equivalent for development.

EPIC E2 — CI/CD

VOS-020 [P0] GitHub Actions PR pipeline

Lint, typecheck, unit tests, build.

VOS-021 [P0] Integration-test service containers

PostgreSQL + Redis.

VOS-022 [P1] E2E pipeline

Run critical web/API tests.

VOS-023 [P1] Container builds

Build web/api/worker images.

VOS-024 [P1] Staging deployment

Automated deployment after main.

VOS-025 [P1] Production deployment gate

Approval + smoke tests + rollback documentation.

VOS-026 [P0] Secret scanning

Prevent accidental credential commits.

EPIC E3 — Authentication and tenant foundation

VOS-030 [P0] User authentication

Login/session flow.

VOS-031 [P0] Tenant entity

Tenant creation and lifecycle.

VOS-032 [P0] Tenant membership

User ↔ tenant roles.

VOS-033 [P0] Tenant context

Resolve active tenant from authenticated request.

VOS-034 [P0] RBAC

Owner/admin/manager/staff/read-only.

VOS-035 [P0] RLS framework

Enable and test PostgreSQL RLS.

VOS-036 [P0] Cross-tenant security tests

Read/write/delete isolation.

EPIC E4 — Business configuration

VOS-040 [P1] Business profile

Name, contact details, timezone, locale.

VOS-041 [P1] Locations

Service/physical locations.

VOS-042 [P1] Services

Service catalog.

VOS-043 [P1] Service areas

Cities/ZIP/service regions.

VOS-044 [P1] Business hours

Weekly availability rules.

VOS-045 [P1] Custom field definitions

Generic vertical-specific fields.

VOS-046 [P1] Custom field values

Lead/contact/estimate values.

EPIC E5 — CRM

VOS-050 [P1] Contacts

CRUD + search + pagination.

VOS-051 [P1] Leads

CRUD + pipeline.

VOS-052 [P1] Lead sources

Google, website, phone, referral, ads, etc.

VOS-053 [P1] Lead state transitions

Validate lifecycle transitions.

VOS-054 [P1] Lead assignment

Owner/staff assignment.

VOS-055 [P1] CRM dashboard

Lead counts and pipeline.

EPIC E6 — Conversations

VOS-060 [P1] Conversation entity

Channel-independent thread.

VOS-061 [P1] Message entity

Inbound/outbound state.

VOS-062 [P1] Messaging provider interface

No provider-specific domain dependency.

VOS-063 [P1] First messaging provider adapter

Select provider during implementation.

VOS-064 [P1] Webhook ingestion

Signature validation + deduplication.

VOS-065 [P1] Unified inbox API

List/view/respond.

EPIC E7 — AI foundation

VOS-070 [P1] LLMProvider interface

Generation/streaming contract.

VOS-071 [P1] First LLM adapter

Select provider during implementation.

VOS-072 [P1] Agent entity/configuration

Versioned agent configuration.

VOS-073 [P1] Tool framework

Typed tool contracts + authorization.

VOS-074 [P1] Business context builder

Structured tenant context.

VOS-075 [P1] Knowledge document model

Tenant knowledge.

VOS-076 [P1] Embedding pipeline

Chunk + embed + persist.

VOS-077 [P1] pgvector retrieval

Tenant-filtered semantic retrieval.

VOS-078 [P1] Agent execution telemetry

Agent/tool/provider metadata.

EPIC E8 — Workflow engine

VOS-080 [P0] Event model

Versioned internal event schema.

VOS-081 [P1] Workflow definition model

Versioned workflows.

VOS-082 [P1] Workflow run model

Persist execution state.

VOS-083 [P1] Workflow worker

BullMQ execution.

VOS-084 [P1] Wait step

Delayed jobs.

VOS-085 [P1] Condition step

Safe condition evaluation.

VOS-086 [P1] Message action

Send through provider abstraction.

VOS-087 [P1] Agent action

Invoke agent.

VOS-088 [P1] Idempotency

Prevent duplicate side effects.

VOS-089 [P1] Retry/failure policy

Bounded retries + escalation.

EPIC E9 — Appointments

VOS-090 [P1] Calendar provider interface

Availability/create/cancel.

VOS-091 [P1] Calendar adapter

First provider.

VOS-092 [P1] Appointment domain

State and validation.

VOS-093 [P1] Booking tool

AI-authorized appointment creation.

VOS-094 [P1] Booking workflow

Lead → qualification → booking.

EPIC E10 — AI Receptionist

VOS-100 [P1] Receptionist agent

Business-grounded conversation.

VOS-101 [P1] Qualification toolset

Service/location/budget/timeline.

VOS-102 [P1] Human handoff

Explicit escalation.

VOS-103 [P1] Receptionist evaluation set

Synthetic regression cases.

VOS-104 [P1] Missed-call workflow

Call missed → AI/SMS → follow-up.

EPIC E11 — Estimates

VOS-110 [P1] Estimate entity

Create/send/status.

VOS-111 [P1] Estimate dashboard

Pending/sent/follow-up.

VOS-112 [P1] Estimate follow-up workflow

Automated reminders.

VOS-113 [P2] Estimate AI assistant

Draft communication only; no autonomous price approval.

EPIC E12 — Audit and observability

VOS-120 [P0] Structured logging

Request/job correlation.

VOS-121 [P0] Audit log

Important state changes.

VOS-122 [P1] Workflow telemetry

Run/step metrics.

VOS-123 [P1] AI telemetry

Provider/model/token/latency where available.

VOS-124 [P1] Error tracking

Centralized exception reporting.

VOS-125 [P1] Operational dashboard

System health and workflow metrics.

EPIC E13 — Remodeling template

VOS-130 [P1] Remodeling services

Kitchen, bathroom, whole-home, additions, etc.

VOS-131 [P1] Remodeling custom fields

Project type, property type, budget, timeline, location.

VOS-132 [P1] Remodeling qualification workflow

VOS-133 [P1] Remodeling default receptionist configuration

VOS-134 [P1] Remodeling seed/demo tenant

EPIC E14 — End-to-end vertical slice

VOS-140 [P0] Critical path E2E

```
Create tenant
→ configure remodeling
→ inbound lead
→ contact
→ lead
→ conversation
→ AI qualification
→ follow-up
→ appointment
→ estimate
→ estimate follow-up
→ audit
```

VOS-141 [P0] Tenant isolation E2E

VOS-142 [P1] Provider failure E2E

Messaging/calendar failure and recovery.

VOS-143 [P1] Worker retry E2E

Ensure idempotent external action.

Definition of MVP complete

V0.1 is complete only when:

• the critical vertical slice works end-to-end;

• tenant isolation is verified;

• CI/CD deploys repeatably;

• AI tool permissions are enforced;

• workflows are retryable/idempotent;

• a remodeling template provisions a usable tenant;

• the first real customer can operate the revenue workflow without engineering intervention for normal cases.

Post-MVP backlog

P2/P3 candidates:

• review automation;

• customer re-engagement;

• AI proposal drafting;

• project/task management;

• invoicing;

• payments;

• advanced analytics;

• multi-location support;

• workflow builder UI;

• additional home-service templates;

• additional communication providers;

• enterprise tenant isolation options.